



طراحی سیستم‌های دیجیتال

بهار ۱۴۰۵

استاد: دکتر اجلالی

محسن صلاح - ۴۰۳۱۰۶۲۳۸

صف هم‌گاه FIFO

تمرین ۵ - سوال ۳

۱ هدف تمرین

در این سؤال، یک صف سنکرون (Synchronous FIFO) با استفاده از یک میان‌گیر چرخشی (Circular Buffer) و آرایه‌ی دو بعدی از ثبات‌ها طراحی و پیاده‌سازی شده است. این مدار هر دو عملیات خواندن و نوشتن را در یک چرخه‌ی ساعت انجام داده و وضعیت پر یا خالی بودن صف را نیز به کمک یک شمارنده‌ی داخلی نگهداری می‌کند.

۲ توضیح معماری

صف سنکرون از بخش‌های اصلی زیر تشکیل شده است:

- حافظه چرخشی (fifo_mem): یک آرایه دو بعدی از ثبات‌ها به ابعاد $\text{DATA_WIDTH} \times \text{FIFO_DEPTH}$.
- اشاره‌گرها (rd_ptr و wr_ptr): ثبات‌های ADDR_WIDTH بیتی که آدرس خانه بعدی برای نوشتن و خواندن را مشخص می‌کنند و با هر عملیات یک واحد افزایش می‌یابند (با سرریز طبیعی آدرس‌ها دور می‌زنند).
- شمارنده عناصر (count): ثبات $\text{ADDR_WIDTH} + 1$ بیتی برای ذخیره‌ی تعداد عناصر موجود در صف (از ۰ تا FIFO_DEPTH).

منطق تشخیص وضعیت پر و خالی بودن مدار به صورت ترکیبی و مستقیم از روی شمارنده محاسبه می‌شود:

```
1 assign empty = (count == 0);  
2 assign full = (count == FIFO_DEPTH);
```

در حالت درخواست همزمان خواندن و نوشتن ($\text{rd_en}=1$ و $\text{wr_en}=1$):

- اگر صف نه خالی باشد و نه پر، هر دو عملیات انجام شده و count تغییر نمی‌کند.
- اگر صف خالی باشد ($\text{empty}=1$)، فقط نوشتن معتبر است و count یک واحد افزایش می‌یابد.
- اگر صف پر باشد ($\text{full}=1$)، فقط خواندن معتبر است و count یک واحد کاهش می‌یابد.

۳ توضیح کد

۱.۳ sync_fifo ماژول

این ماژول عملیات بازنشانی سنکرون، به‌روزرسانی اشاره‌گرها، نوشتن در حافظه، خواندن روی خروجی dout و مدیریت شمارنده count را در لبه بالارونده کلک انجام می‌دهد. کد اصلی مدار به این صورت می‌باشد:

```
1 always @(posedge clk) begin
2     if (rst) begin
3         wr_ptr <= {ADDR_WIDTH{1'b0}};
4         rd_ptr <= {ADDR_WIDTH{1'b0}};
5         count <= {(ADDR_WIDTH+1){1'b0}};
6         dout <= {DATA_WIDTH{1'b0}};
7     end else begin
8         // Write Operation
9         if ((wr_en && !full) || (wr_en && rd_en && empty)) begin
10             fifo_mem[wr_ptr] <= din;
11             wr_ptr <= wr_ptr + 1'b1;
12         end
13
14         // Read Operation
15         if ((rd_en && !empty) || (wr_en && rd_en && full)) begin
16             dout <= fifo_mem[rd_ptr];
17             rd_ptr <= rd_ptr + 1'b1;
18         end
19
20         // Counter Logic
21         case ({wr_en, rd_en})
22             2'b10: if (!full) count <= count + 1'b1; // Write only
23             2'b01: if (!empty) count <= count - 1'b1; // Read only
24             2'b11: begin // Simultaneous R/W
25                 if (empty) count <= count + 1'b1;
26                 else if (full) count <= count - 1'b1;
27             end
28             default: ;
29         endcase
30     end
31 end
```

۲.۳ تست‌بنچ

تسک **check_step**: این تسک پس از لبه‌ی فعال ساعت با یک واحد تأخیر (#1) مقادیر سیگنال‌های ورودی (rd_en و wr_en، din، rst) و مقادیر مورد انتظار را در ترمینال چاپ کرده و در صورت عدم تطابق، متغیر tc_failed را یک می‌کند:

```
1 task check_step;
2     input [DATA_WIDTH-1:0] exp_dout;
3     input exp_full, exp_empty;
4     begin
```

```

5      #1; // Delay for non-blocking assignments
6      $display("%%[Cycle_%02d]%%IN:rst=%b%%din=0x%02h%%wr=%b%%rd=%b%%DUT:dout
              =0x%02h%%full=%b%%empty=%b%%EXP:dout=0x%02h%%full=%b%%empty=%b",
7              cycle_num, rst, din, wr_en, rd_en, dout, full, empty, exp_dout,
              exp_full, exp_empty);
8
9      if (dout != exp_dout || full != exp_full || empty != exp_empty)
10         begin
11             $display("%%%%%%%%[ERROR]Mismatch at cycle_%02d!", cycle_num);
12             tc_failed = 1;
13         end
14     endtask

```

تست‌های کمک‌کننده: تست‌بنچ از ۴ تست اصلی برای اعمال ورودی‌ها روی لبه‌ی پایین کلک (negedge clk) و ارزیابی خروجی در لبه‌ی بالایی بعد استفاده می‌کند:

- apply_reset: سیگنال rst=1 را اعمال کرده و پس از یک سیکل، خالی بودن صف را ارزیابی می‌کند.
- do_write: مقدار ورودی din را قرار داده، wr_en=1 می‌کند و صحّت خروجی‌ها را بررسی می‌نماید.
- do_read: سیگنال rd_en=1 را فعال کرده و داده‌ی خوانده‌شده روی dout را ارزیابی می‌کند.
- do_both: هر دو سیگنال wr_en=1 و rd_en=1 را همزمان فعال کرده و رفتار مدار در شرایط مختلف را می‌سنجد.

تست‌کیس‌ها:

تست‌های اصلی به ترتیب زیر اجرا می‌شوند:

```

1 // Test 1: Synchronous Reset
2 tc_failed = 0;
3 apply_reset();
4 finish_test_case("Synchronous_Reset");

```

با اعمال rst در لبه‌ی فعال ساعت، تمامی اشاره‌گرها و شمارنده‌ی داخلی صفر شده و بررسی می‌شود که empty=1 و full=0 باشد.

```

1 // Test 2: Sequential Writes and Reads (FIFO ordering)
2 tc_failed = 0;
3 apply_reset();
4 do_write(8'hA1, 8'h00, 1'b0, 1'b0);
5 do_write(8'hA2, 8'h00, 1'b0, 1'b0);
6 do_write(8'hA3, 8'h00, 1'b0, 1'b0);
7 do_write(8'hA4, 8'h00, 1'b0, 1'b0);
8 do_read(8'hA1, 1'b0, 1'b0);
9 do_read(8'hA2, 1'b0, 1'b0);
10 do_read(8'hA3, 1'b0, 1'b0);
11 do_read(8'hA4, 1'b0, 1'b1);
12 finish_test_case("Sequential_Write_Read_FIFO_Order");

```

۴ داده‌ی متوالی در صف نوشته شده و سپس به ترتیب خوانده می‌شوند تا حفظ ترتیب ورود و خروج (FIFO order) و فعال شدن empty=1 پس از خواندن آخرین داده تأیید شود.

```
1 // Test 3: Fill Completely + Write-to-Full Ignored
2 tc_failed = 0;
3 apply_reset();
4 for (i = 0; i < FIFO_DEPTH; i = i + 1)
5     do_write(8'h10 + i, 8'h00, (i == FIFO_DEPTH - 1), 1'b0);
6 do_write(8'hFF, 8'h00, 1'b1, 1'b0); // Sentinel write to full FIFO
7 do_read(8'h10, 1'b0, 1'b0); // Verify head data remains 0x10
8 finish_test_case("Fill_Completely_Write-to-Full_Ignored");
```

صف به میزان FIFO_DEPTH خانه پر شده تا full=1 گردد. سپس یک داده‌ی اضافی (0xFF) روی صف پر نوشته می‌شود؛ در مرحله‌ی بعد با خواندن اولین عنصر بررسی می‌شود که داده‌ی اضافی نادیده گرفته شده و داده‌ی اصلی سر صف تغییر نکرده باشد.

```
1 // Test 4: Drain Completely + Read-from-Empty Ignored
2 tc_failed = 0;
3 apply_reset();
4 for (i = 0; i < FIFO_DEPTH; i = i + 1)
5     do_write(8'h20 + i, 8'h00, (i == FIFO_DEPTH - 1), 1'b0);
6 for (i = 0; i < FIFO_DEPTH; i = i + 1)
7     do_read(8'h20 + i, 1'b0, (i == FIFO_DEPTH - 1));
8 do_read(8'h20 + FIFO_DEPTH - 1, 1'b0, 1'b1); // Spurious read from empty
9 finish_test_case("Drain_Completely_Read-from-Empty_Ignored");
```

صف ابتدا کاملاً پر شده و سپس تمام داده‌های آن خوانده می‌شوند تا empty=1 گردد. درخواست خواندن مجدد روی صف خالی اعمال شده و بررسی می‌شود که عملیات نادیده گرفته شده و empty=1 باقی بماند.

```
1 // Test 5: Simultaneous RW: Normal State
2 tc_failed = 0;
3 apply_reset();
4 for (i = 0; i < 8; i = i + 1)
5     do_write(8'h30 + i, 8'h00, 1'b0, 1'b0);
6 do_both(8'h40, 8'h30, 1'b0, 1'b0);
7 do_both(8'h41, 8'h31, 1'b0, 1'b0);
8 do_both(8'h42, 8'h32, 1'b0, 1'b0);
9 do_both(8'h43, 8'h33, 1'b0, 1'b0);
10 finish_test_case("Simultaneous_RW_Normal_State");
```

در حالت نیمه‌پر، عملیات خواندن و نوشتن همزمان (do_both) اجرا می‌شود. صحت خروجی داده‌ی خوانده‌شده بررسی شده و تأیید می‌گردد که تعداد عناصر موجود در صف (count) تغییر نکرده است.

```
1 // Test 6: Simultaneous RW: FIFO Empty
2 tc_failed = 0;
3 apply_reset();
4 do_both(8'hAB, 8'h00, 1'b0, 1'b0); // Simultaneous RW on empty FIFO
```

```
5 do_read(8'hAB, 1'b0, 1'b1); // Read back newly written byte
6 finish_test_case("Simultaneous_RW:_FIFO_Empty");
```

روی صف خالی درخواست همزمان خواندن و نوشتن داده می‌شود. طبق منطق مدار، فقط عملیات نوشتن باید معتبر باشد؛ بلافاصله با یک خواندن مجدد، دریافت داده‌ی جدید (0xAB) ارزیابی می‌گردد.

```
1 // Test 7: Simultaneous RW: FIFO Full
2 tc_failed = 0;
3 apply_reset();
4 for (i = 0; i < FIFO_DEPTH; i = i + 1)
5     do_write(8'h50 + i, 8'h00, (i == FIFO_DEPTH - 1), 1'b0);
6 do_both(8'hFF, 8'h50, 1'b0, 1'b0); // Simultaneous RW on full FIFO
7 do_read(8'h51, 1'b0, 1'b0); // Confirm next item intact (0xFF was ignored)
8 finish_test_case("Simultaneous_RW:_FIFO_Full");
```

روی صف پر درخواست همزمان خواندن و نوشتن اعمال می‌شود. در این حالت فقط خواندن معتبر است؛ با بررسی داده‌ی خوانده‌شده بعدی (0x51) تأیید می‌شود که داده‌ی جدید (0xFF) نوشتن نشده است.

۴ اجرای کد

نتیجه‌ی اجرای کد به این صورت می‌باشد:

```
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HWS_Mohsen_Salah_403106238/Q3
/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HWS_Mohsen_Salah_403106238/Q3

mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HWS_Mohsen_Salah_403106238/Q3$ iverilog DSD_HWS_Q3.v DSD_HWS_Q3_tb.v && vvp a.out
VCD info: dumpfile DSD_HWS_Q3.vcd opened for output.

=== Synchronous FIFO Testbench ===
[Cycle 01] IN: rst=1 din=0x00 wr=0 rd=0 | DUT: dout=0x00 full=0 empty=1 | EXP: dout=0x00 full=0 empty=1
[PASS] Test 1 (Synchronous Reset): dout=0x00, full=0, empty=1
-----
[Cycle 02] IN: rst=1 din=0x00 wr=0 rd=0 | DUT: dout=0x00 full=0 empty=1 | EXP: dout=0x00 full=0 empty=1
[Cycle 03] IN: rst=0 din=0xa1 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 04] IN: rst=0 din=0xa2 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 05] IN: rst=0 din=0xa3 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 06] IN: rst=0 din=0xa4 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 07] IN: rst=0 din=0xa4 wr=0 rd=1 | DUT: dout=0xa1 full=0 empty=0 | EXP: dout=0xa1 full=0 empty=0
[Cycle 08] IN: rst=0 din=0xa4 wr=0 rd=1 | DUT: dout=0xa2 full=0 empty=0 | EXP: dout=0xa2 full=0 empty=0
[Cycle 09] IN: rst=0 din=0xa4 wr=0 rd=1 | DUT: dout=0xa3 full=0 empty=0 | EXP: dout=0xa3 full=0 empty=0
[Cycle 10] IN: rst=0 din=0xa4 wr=0 rd=1 | DUT: dout=0xa4 full=0 empty=0 | EXP: dout=0xa4 full=0 empty=0
[PASS] Test 2 (Sequential Write / Read / FIFO Order): dout=0xa4, full=0, empty=1
-----
[Cycle 11] IN: rst=1 din=0x00 wr=0 rd=0 | DUT: dout=0x00 full=0 empty=1 | EXP: dout=0x00 full=0 empty=1
[Cycle 12] IN: rst=0 din=0x10 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 13] IN: rst=0 din=0x11 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 14] IN: rst=0 din=0x12 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 15] IN: rst=0 din=0x13 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 16] IN: rst=0 din=0x14 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 17] IN: rst=0 din=0x15 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 18] IN: rst=0 din=0x16 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 19] IN: rst=0 din=0x17 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 20] IN: rst=0 din=0x18 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 21] IN: rst=0 din=0x19 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 22] IN: rst=0 din=0x1a wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 23] IN: rst=0 din=0x1b wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 24] IN: rst=0 din=0x1c wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 25] IN: rst=0 din=0x1d wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 26] IN: rst=0 din=0x1e wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 27] IN: rst=0 din=0x1f wr=1 rd=0 | DUT: dout=0x00 full=1 empty=0 | EXP: dout=0x00 full=1 empty=0
[Cycle 28] IN: rst=0 din=0xff wr=1 rd=0 | DUT: dout=0x00 full=1 empty=0 | EXP: dout=0x00 full=1 empty=0
[Cycle 29] IN: rst=0 din=0xff wr=0 rd=1 | DUT: dout=0x10 full=0 empty=0 | EXP: dout=0x10 full=0 empty=0
[PASS] Test 3 (Fill Completely + Write-to-Full Ignored): dout=0x10, full=0, empty=0
-----
```

شکل ۱: خروجی تست‌بنچ در ترمینال

```
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HWS_Mohsen_Salah_403106238/Q3
/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HWS_Mohsen_Salah_403106238/Q3

[Cycle 30] IN: rst=1 din=0x00 wr=0 rd=0 | DUT: dout=0x00 full=0 empty=1 | EXP: dout=0x00 full=0 empty=1
[Cycle 31] IN: rst=0 din=0x20 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 32] IN: rst=0 din=0x21 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 33] IN: rst=0 din=0x22 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 34] IN: rst=0 din=0x23 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 35] IN: rst=0 din=0x24 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 36] IN: rst=0 din=0x25 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 37] IN: rst=0 din=0x26 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 38] IN: rst=0 din=0x27 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 39] IN: rst=0 din=0x28 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 40] IN: rst=0 din=0x29 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 41] IN: rst=0 din=0x2a wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 42] IN: rst=0 din=0x2b wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 43] IN: rst=0 din=0x2c wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 44] IN: rst=0 din=0x2d wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 45] IN: rst=0 din=0x2e wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 46] IN: rst=0 din=0x2f wr=1 rd=0 | DUT: dout=0x00 full=1 empty=0 | EXP: dout=0x00 full=1 empty=0
[Cycle 47] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x20 full=0 empty=0 | EXP: dout=0x20 full=0 empty=0
[Cycle 48] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x21 full=0 empty=0 | EXP: dout=0x21 full=0 empty=0
[Cycle 49] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x22 full=0 empty=0 | EXP: dout=0x22 full=0 empty=0
[Cycle 50] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x23 full=0 empty=0 | EXP: dout=0x23 full=0 empty=0
[Cycle 51] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x24 full=0 empty=0 | EXP: dout=0x24 full=0 empty=0
[Cycle 52] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x25 full=0 empty=0 | EXP: dout=0x25 full=0 empty=0
[Cycle 53] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x26 full=0 empty=0 | EXP: dout=0x26 full=0 empty=0
[Cycle 54] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x27 full=0 empty=0 | EXP: dout=0x27 full=0 empty=0
[Cycle 55] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x28 full=0 empty=0 | EXP: dout=0x28 full=0 empty=0
[Cycle 56] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x29 full=0 empty=0 | EXP: dout=0x29 full=0 empty=0
[Cycle 57] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x2a full=0 empty=0 | EXP: dout=0x2a full=0 empty=0
[Cycle 58] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x2b full=0 empty=0 | EXP: dout=0x2b full=0 empty=0
[Cycle 59] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x2c full=0 empty=0 | EXP: dout=0x2c full=0 empty=0
[Cycle 60] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x2d full=0 empty=0 | EXP: dout=0x2d full=0 empty=0
[Cycle 61] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x2e full=0 empty=0 | EXP: dout=0x2e full=0 empty=0
[Cycle 62] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x2f full=0 empty=1 | EXP: dout=0x2f full=0 empty=1
[Cycle 63] IN: rst=0 din=0x2f wr=0 rd=1 | DUT: dout=0x2f full=0 empty=1 | EXP: dout=0x2f full=0 empty=1
[PASS] Test 4 (Drain Completely + Read-from-Empty Ignored): dout=0x2f, full=0, empty=1
-----
```

شکل ۲: خروجی تست‌بنچ در ترمینال (ادامه)

```

mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HW5_Mohsen_Salah_403106238/Q3
/mnt/Q/SUT/Term_4/DSD/HW/HW-5/DSD_HW5_Mohsen_Salah_403106238/Q3

[Cycle 64] IN: rst=1 din=0x00 wr=0 rd=0 | DUT: dout=0x00 full=0 empty=1 | EXP: dout=0x00 full=0 empty=1
[Cycle 65] IN: rst=0 din=0x30 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 66] IN: rst=0 din=0x31 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 67] IN: rst=0 din=0x32 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 68] IN: rst=0 din=0x33 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 69] IN: rst=0 din=0x34 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 70] IN: rst=0 din=0x35 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 71] IN: rst=0 din=0x36 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 72] IN: rst=0 din=0x37 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 73] IN: rst=0 din=0x40 wr=1 rd=1 | DUT: dout=0x30 full=0 empty=0 | EXP: dout=0x30 full=0 empty=0
[Cycle 74] IN: rst=0 din=0x41 wr=1 rd=1 | DUT: dout=0x31 full=0 empty=0 | EXP: dout=0x31 full=0 empty=0
[Cycle 75] IN: rst=0 din=0x42 wr=1 rd=1 | DUT: dout=0x32 full=0 empty=0 | EXP: dout=0x32 full=0 empty=0
[Cycle 76] IN: rst=0 din=0x43 wr=1 rd=1 | DUT: dout=0x33 full=0 empty=0 | EXP: dout=0x33 full=0 empty=0
[PASS] Test 5 (Simultaneous RW: Normal State): dout=0x33, full=0, empty=0

[Cycle 77] IN: rst=1 din=0x00 wr=0 rd=0 | DUT: dout=0x00 full=0 empty=1 | EXP: dout=0x00 full=0 empty=1
[Cycle 78] IN: rst=0 din=0xab wr=1 rd=1 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 79] IN: rst=0 din=0xab wr=0 rd=1 | DUT: dout=0xab full=0 empty=1 | EXP: dout=0xab full=0 empty=1
[PASS] Test 6 (Simultaneous RW: FIFO Empty): dout=0xab, full=0, empty=1

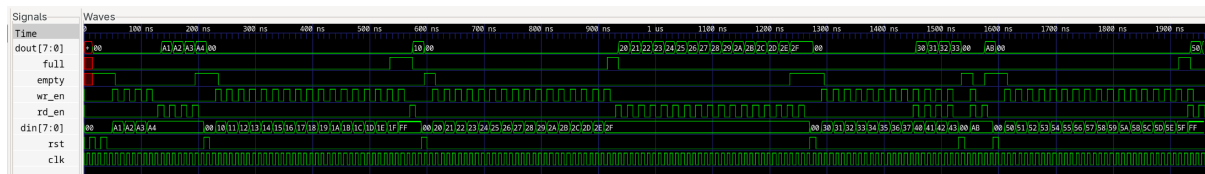
[Cycle 80] IN: rst=1 din=0x00 wr=0 rd=0 | DUT: dout=0x00 full=0 empty=1 | EXP: dout=0x00 full=0 empty=1
[Cycle 81] IN: rst=0 din=0x50 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 82] IN: rst=0 din=0x51 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 83] IN: rst=0 din=0x52 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 84] IN: rst=0 din=0x53 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 85] IN: rst=0 din=0x54 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 86] IN: rst=0 din=0x55 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 87] IN: rst=0 din=0x56 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 88] IN: rst=0 din=0x57 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 89] IN: rst=0 din=0x58 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 90] IN: rst=0 din=0x59 wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 91] IN: rst=0 din=0x5a wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 92] IN: rst=0 din=0x5b wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 93] IN: rst=0 din=0x5c wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 94] IN: rst=0 din=0x5d wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 95] IN: rst=0 din=0x5e wr=1 rd=0 | DUT: dout=0x00 full=0 empty=0 | EXP: dout=0x00 full=0 empty=0
[Cycle 96] IN: rst=0 din=0x5f wr=1 rd=0 | DUT: dout=0x00 full=1 empty=0 | EXP: dout=0x00 full=1 empty=0
[Cycle 97] IN: rst=0 din=0xff wr=1 rd=1 | DUT: dout=0x50 full=0 empty=0 | EXP: dout=0x50 full=0 empty=0
[Cycle 98] IN: rst=0 din=0xff wr=0 rd=1 | DUT: dout=0x51 full=0 empty=0 | EXP: dout=0x51 full=0 empty=0
[PASS] Test 7 (Simultaneous RW: FIFO Full): dout=0x51, full=0, empty=0

7 PASSED, 0 FAILED out of 7 tests.
DSD_HW5_Q3_tb.v:195: $finish called at 1960000 (1ps)
mohsen@Quanta:/mnt/D/SUT/Term_4/DSD/HW/HW-5/DSD_HW5_Mohsen_SaLah_403106238/Q3$

```

شکل ۳: خروجی تست بنچ در ترمینال (ادامه)

همان طور که مشاهده می‌شود، تمامی ۷ تست‌کیس با موفقیت PASS شده‌اند. شکل موج سیگنال‌ها در GTKWave نیز به این صورت می‌باشد:



شکل ۴: شکل موج سیگنال‌ها در GTKWave

همان طور که مشاهده می‌شود، تغییرات اشاره‌گرها، سیگنال‌های full و empty و داده‌های خروجی مطابق با رفتار مورد انتظار مدار عمل می‌کنند.